

Virtual Memory and Secondary Storage

Lecture Notes — Week 10

PCC CS-401: Computer Organization and Architecture

September 5, 2026

10.1 The Hierarchy Continues: From Cache to Storage

Week 9 placed a small, fast cache between the CPU and a larger, slower main memory. A cache line is found by mapping an address and checking a tag; a miss fetches a block and may trigger replacement. The design goal was not to make all memory fast, but to make the common case fast by exploiting locality.

This week asks whether the same idea can operate one level lower. If a program's address space is larger than physical main memory, can a small, fast level (DRAM) stand in for a larger, slower level (secondary storage)? The answer requires six connected ideas:

1. content-addressable lookup through associative memory and CAM;
2. virtual addresses, page tables, and TLB translation;
3. page faults and demand paging when a page is not resident;
4. segmentation, partitioning, fragmentation, and allocation policies;
5. HDDs, optical media, SSDs, and RAID as storage technologies; and
6. a decision procedure that connects lookup, placement, failure response, and cost across Unit III.

The running system has a 16-bit virtual address, 256-byte pages, and 8 physical frames. We first translate virtual address 0x2F34, then follow the page-fault and storage consequences of that translation.

10.2 Associative Lookup and CAM

10.2.1 When Content Is the Lookup Key

The cache question from Week 9 was: “Is the block with this tag present?” Under fully associative mapping, the tag may be in any cache line, so checking one predetermined address is insufficient. A conventional RAM performs **address** $\rightarrow$ **contents**; an associative lookup performs **content pattern** $\rightarrow$ **matching row** by comparing a search key against many stored entries in parallel. This is the standard CAM hardware treatment of associative cache lookup (see Stallings, 11e, §5.2, PDF pp. 193–194 [2, §5.2, PDF pp. 193–194]).

Definition (Associative memory). *Associative memory is memory searched by the content pattern being sought, rather than by one fixed address. Parallel comparison makes placement flexible, but requires multiple comparators, match-selection logic, wiring, area, energy, and verification effort.*

10.2.2 A CAM Lookup

Content-addressable memory (CAM) compares the search key with every stored row at once. The match vector identifies the row whose contents equal the key; the row’s associated data can then be read. In a TLB, the compared content is typically a virtual page number together with address-space information, and the matching entry supplies a physical frame number.

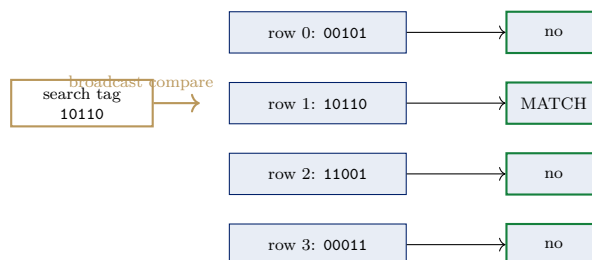


Figure 10.1: CAM compares one search tag with all rows in parallel and emits a match vector.

The benefit is flexible placement: a key need not be assigned to one predetermined row, reducing conflict misses. The cost is that all comparisons and the match-selection logic operate in parallel. This is the same placement-versus-hardware trade-off behind fully associative caches and TLBs. Associative does not mean “random”; it describes the lookup method.

10.3 Virtual Memory, Paging, and Address Translation

10.3.1 A Private Address Space Backed by DRAM and Storage

A process should be able to use a simple, contiguous address space without knowing which physical locations are free or which other process owns them. A **virtual address** is generated by the CPU for the process. The memory-management unit (MMU) translates it to a **physical address** in main memory. Pages are the virtual-memory analogue of cache blocks, and physical frames are the fixed-size slots that hold them. If the page is absent, the hardware raises a **page fault**; the operating system can fetch the page from secondary storage (Patterson and Hennessy, ARM 5e, §5.7, p. 441 [1, §5.7, p. 441]).

Definition (Paging). *Paging divides a virtual address space into fixed-size pages and physical memory into equal-size frames. Translation replaces the virtual page number with a physical frame number while preserving the offset within the page.*

10.3.2 The Address Fields

For the running system, the virtual address width is $V = 16$ bits and the page size is $256 = 2^8$ bytes. Physical memory has 8 frames, or 2^3 frames. Therefore:

- the virtual address space has $2^{16} = 65,536$ bytes = 64 KiB;
- the number of virtual pages is $2^{16}/2^8 = 2^8 = 256$;
- physical memory has $8 \times 256 = 2,048$ bytes = 2 KiB;
- a virtual address contains an 8-bit virtual page number (VPN) and an 8-bit page offset; and
- a physical address contains a 3-bit physical frame number (PFN) and the same 8-bit offset.

The key invariant is that translation changes only the page/frame field. The offset within a page is never changed.

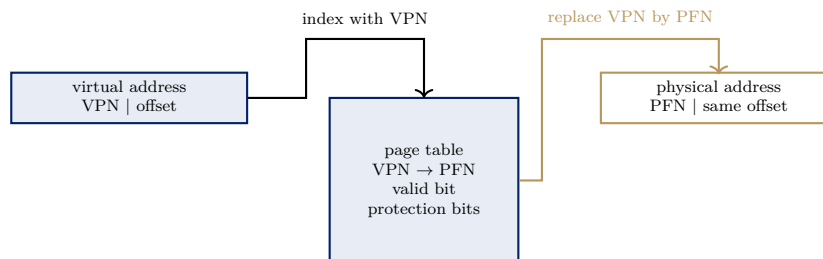


Figure 10.2: Paging translation indexes the page table with the VPN, replaces the VPN by a PFN, and preserves the page offset.

The page table records the current physical frame, validity, and access permissions for each virtual page. A valid entry gives a resident mapping; an invalid entry signals that the operating system must intervene.

Example 10.3.1 (Translating $0x2F34$). *First write the virtual address in binary:*

$$0x2F34 = 0010\ 1111\ 0011\ 0100_2.$$

The eight high bits are the VPN and the eight low bits are the offset:

$$\underbrace{0010\ 1111}_{\text{VPN}=0x2F=47} \quad \underbrace{0011\ 0100}_{\text{offset}=0x34=52}.$$

Suppose the page-table entry for VPN 47 is valid and maps it to physical frame 5, whose three-bit representation is 101_2 . The physical address is

$$5 \times 256 + 52 = 1,332 \text{ bytes.}$$

Equivalently, concatenate the frame field and the unchanged offset:

$$0x5 \parallel 0x34 = 0x534.$$

The low eight bits stayed $0x34$; only the page number changed.

10.3.3 The TLB: A Cache of Translations

Definition (Translation Lookaside Buffer). *A TLB is a small associative cache of recent page-table mappings, from VPN to PFN. It is checked before the larger page table (Patterson and Hennessy, ARM 5e, §5.7, p. 441 [1, §5.7, p. 441]).*

The translation path is:

1. split the CPU's virtual address into VPN and offset;
2. compare the VPN with stored TLB entries in parallel;
3. on a TLB hit, obtain the PFN, concatenate it with the offset, and access DRAM; or
4. on a TLB miss, consult the page table. If its entry is valid, fill the TLB and retry; if it is invalid, take a page fault.

The TLB is associative because the desired VPN may occupy any TLB entry; its match is a content lookup rather than an indexed lookup at one fixed location.

10.4 Page Faults, Segmentation, and Allocation

10.4.1 Demand Paging

Definition (Page fault). *A page fault occurs when translation identifies a virtual page that is not currently resident in a physical frame. It is not automatically a program error: the operating system may make the reference legal by loading the page from storage.*

The fault-handling sequence is explicit:

1. hardware traps to the operating-system page-fault handler;
2. the handler checks the access. An illegal address or permission failure terminates the process, while a legal nonresident page is located in storage;
3. if no free frame exists, the system chooses a victim page, writes it back if it is dirty, and reads the requested page into that frame;
4. the page table and TLB are updated, the requested page is marked valid, and the faulting instruction is restarted.

This is **demand paging**: storage is a backing level, not a substitute for DRAM's latency. The memory-management treatment is indexed in Stallings, 11e, §9.3, PDF pp. 376–389.

10.4.2 Why Page Size Is a Trade-off

Large pages reduce the number of page-table entries and can transfer data efficiently when spatial locality is strong. They also waste more space when only a small part of a page is used and transfer more irrelevant data on a fault. Small pages reduce internal waste and provide finer protection, but require larger page tables and create more translation overhead. A useful page size balances page-table cost, transfer cost, fragmentation, and the program's locality.

10.4.3 Segmentation

Definition (Segmentation). *Segmentation represents an address space as variable-sized logical regions, such as code, data, stack, or a shared library. A logical address consists of a segment number and an offset.*

A segment-table entry supplies a base address, a limit, and access permissions. Translation first checks

$$0 \leq \text{offset} < \text{limit},$$

then forms

$$\text{physical address} = \text{base} + \text{offset}.$$

Protection and sharing can follow program structure: code can be read/execute, while data can be read/write. The variable sizes make external fragmentation possible; paging avoids external fragmentation by using fixed-size frames. Stallings treats segmentation as part of memory management in §9.3, PDF pp. 376–389; Hamacher Ch. 8 is the syllabus-level anchor for the same standard treatment.

Scheme	Allocation unit	Strength	Main cost
Paging	Fixed-size page/frame	Simple placement; no external fragmentation; easy swapping	Internal fragmentation; page-table/TLB overhead
Segmentation	Variable-size logical segment	Natural protection/sharing; programmer-visible units	External fragmentation; compaction or complex allocation
Paged segmentation	Segments divided into pages	Combines logical protection with fixed-size placement	Two-level translation and more metadata

The exam habit is to state the unit first. Fixed-size units imply pages, frames, and internal fragmentation. Variable-size units imply segments, holes, and external fragmentation.

10.4.4 Contiguous Allocation and Hole Selection

Suppose free memory contains holes of 100, 500, 200, 300, and 600 KiB, and a process requests 212 KiB. The three policies choose different holes:

- **First fit** chooses the first adequate hole, 500 KiB, leaving $500 - 212 = 288$ KiB.
- **Best fit** chooses the smallest adequate hole, 300 KiB, leaving $300 - 212 = 88$ KiB, but may create many tiny holes.
- **Worst fit** chooses the largest hole, 600 KiB, leaving $600 - 212 = 388$ KiB and preserving medium-sized holes elsewhere.

In every case, the selected hole is split into the allocated block and a smaller free hole.

Example 10.4.1 (Comparing the three policies). *For the hole list (100, 500, 200, 300, 600) KiB and request 212 KiB, the post-placement free lists are:*

- *first fit:* (100, 288, 200, 300, 600) KiB;
- *best fit:* (100, 500, 200, 88, 600) KiB; and
- *worst fit:* (100, 500, 200, 300, 388) KiB.

The policy choice is therefore a choice about the shape of future holes, not just about where this one process is placed.

10.4.5 Fragmentation and Policy Choice

Definition (Internal and external fragmentation). *Internal fragmentation* is unused space inside an allocated fixed-size unit, such as the unused tail of a page or frame. **External fragmentation** occurs when enough total free space exists, but it is divided into non-contiguous holes and cannot satisfy one contiguous request.

Paging largely removes external fragmentation because any free frame can hold any page, but it does not remove internal fragmentation: the last page of a process is often only partly occupied. Segmentation matches logical units well, but allocation must manage holes; compaction moves allocated segments and is expensive.

For a real-time allocator receiving many requests between 90 and 120 KiB, first fit is a defensible default because it usually searches less. Best fit is defensible only if measurements show that preserving larger holes is worth the extra search and small-hole production. Worst fit is defensible only if the workload benefits from retaining useful medium-sized holes. The engineering answer names the workload and metric, rather than naming a policy in isolation.

10.5 Secondary Storage

10.5.1 HDD Access and Geometry

An HDD stores bits magnetically on rotating platters. A request selects a surface and track, waits for the desired sector to rotate under the head, and then transfers the data. Its access time is decomposed as

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}} + T_{\text{controller}}.$$

Seek time moves the actuator to the target track. **Rotational latency** waits for the target sector; average latency is approximately half a rotation. **Transfer time** reads or writes the sector once positioned. Stallings treats these components in §7.1, PDF p. 265. Sequential access amortizes seek and rotation, whereas random page faults can be dominated by positioning time.

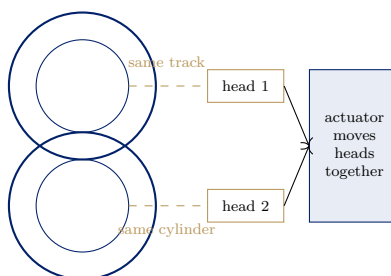


Figure 10.3: HDD geometry: the actuator moves heads together, so corresponding tracks across platter surfaces form a cylinder.

A **cylinder** is the set of tracks at the same radius across surfaces. Keeping related blocks nearby reduces movement and improves effective throughput; this is storage-level spatial locality.

10.5.2 RAID: Parallelism and Recovery

Definition (RAID). *RAID (Redundant Array of Independent Disks) combines multiple physical disks and presents a logical storage system. Its design dimensions are striping for parallelism, mirroring or parity for recovery, and the number of disks left available after redundancy.*

Level	Idea	Trade-off
RAID 0	Block striping, no redundancy	High throughput; one disk failure loses the array
RAID 1	Mirroring	Simple recovery; about half raw capacity
RAID 5	Striping with distributed single parity	One-disk failure tolerance; parity-write cost
RAID 6	Striping with distributed dual parity	Two-disk failure tolerance; more parity overhead

RAID levels and their performance/reliability trade-offs are indexed in Stallings, 11e, §7.2, PDF pp. 276–288, and Patterson and Hennessy, ARM 5e, §5.11, p. 481.

Example 10.5.1 (RAID 5 parity recovery). *Suppose corresponding bytes on three disks are*

$$D_0 = 1010, \quad D_1 = 0110, \quad D_2 = 1100.$$

RAID 5 stores the parity byte

$$P = D_0 \oplus D_1 \oplus D_2.$$

First,

$$1010 \oplus 0110 = 1100, \quad 1100 \oplus 1100 = 0000,$$

so $P = 0000$. If disk D_1 fails, XOR the surviving data and parity:

$$D_1 = D_0 \oplus D_2 \oplus P = 1010 \oplus 1100 \oplus 0000 = 0110.$$

The missing block is recovered exactly. One failure is recoverable because parity preserves the XOR relation; two simultaneous failures require more independent parity, as in RAID 6.

10.5.3 Optical Media and Workload Choice

Optical media encode information in a spiral track of pits and lands. A laser reads changing reflectivity and the drive converts that signal into bits. Common capacities are approximately 700 MB for a CD, 4.7 GB for a single-layer, single-sided DVD, and 25 GB for a single-layer Blu-ray. The shorter-wavelength blue-violet laser and tighter track geometry allow Blu-ray to store more data. Read-only, recordable, and rewritable variants differ in how the recording layer changes. Optical media are removable and inexpensive per disc, but slower and less convenient for frequent updates than SSDs. This is standard secondary-storage treatment; Stallings indexes CD/DVD/Blu-ray in §7.4, PDF p. 292.

The medium should follow the bottleneck:

Medium	Strength	Weakness	Good fit
HDD	Low cost per bit; large capacity	Mechanical seek and rotation	Bulk data, sequential files, cold storage
SSD/flash	Low latency; no moving head	Write/erase limits; higher cost per bit	OS/apps, random access, active working sets
Optical	Removable; useful for distribution/archive	Slow updates; lower capacity per medium	Media distribution and offline copies
RAID array	Parallelism plus optional redundancy	More controllers, disks, parity or mirror overhead	Servers needing throughput and availability

The design question is whether the bottleneck is latency, throughput, capacity, failure recovery, or cost. The answer follows from that bottleneck.

10.6 Unit III Synthesis and Exam Procedure

Across the hierarchy, the same four-part design pattern recurs:

1. identify the **unit**: line, page, segment, block, or stripe;

2. identify **lookup/placement**: tag comparison, page table/TLB, hole selection, disk geometry, or RAID mapping;
3. state the **miss/failure response**: fetch a block, handle a page fault, split or compact a hole, seek and transfer, or reconstruct from redundancy; and
4. state the **cost**: hardware, latency, capacity waste, fragmentation, bandwidth, or redundancy overhead.

Week 9 made DRAM appear faster through cache. Week 10 makes storage-backed capacity appear like a larger private memory through virtual memory, while page faults expose the true cost. No level removes the cost; it changes when and where the cost is paid.

When a program repeatedly touches a small set of pages but both its TLB miss rate and page-fault rate are high, investigate four facts: TLB capacity and associativity may be too small for the active VPNs; too few physical frames may be causing repeated faults and thrashing; page size may be fetching irrelevant data or creating excessive metadata; and an access pattern that alternates across many pages may defeat replacement even when total memory seems large.

For an exam response, translate an address by splitting VPN and offset, replacing only the VPN and preserving the offset. For a page fault, state the trap, validity/protection check, victim selection, dirty write-back, page-in, page-table/TLB update, and instruction restart. For paging versus segmentation, name the unit and distinguish internal from external fragmentation. For an allocation policy, show the hole list before and after placement. For an HDD, decompose seek, rotational latency, transfer, and controller time. For RAID, distinguish striping, mirroring, parity, capacity, throughput, and tolerated failures. For associative memory, explain parallel comparison and its hardware cost, then connect CAM/TLB lookup to fully associative cache mapping.

Exercises

1. A system has a 20-bit virtual address, 1 KiB pages, and 16 physical frames. Split virtual address `0xABCDE` into VPN and offset, and state the width of each physical-address field. Assume the VPN maps to frame 9 and compute the physical address.
2. For holes of (120, 280, 640, 200) KiB and a request of 190 KiB, show the resulting free list under first fit, best fit, and worst fit. Identify which policy leaves the smallest remainder in the selected hole.
3. Given $D_0 = 1101$, $D_1 = 0011$, and $D_2 = 0101$, compute RAID 5 parity, then reconstruct D_2 after it fails.
4. A workload alternates among more VPNs than the TLB can hold, while all referenced pages fit in physical memory. Diagnose why the TLB miss rate can be high while the page-fault rate remains low, and state one hardware fact and one access-pattern fact to investigate.

Solutions

1. A 1 KiB page has 2^{10} bytes, so the offset is 10 bits and the 20-bit VPN is the high 10 bits. Since `0xABCDE` is $1010\ 1011\ 1100\ 1101\ 1110_2$, its low 10 bits are $00\ 1101\ 1110_2 = 0x0DE$ and its VPN is $1010\ 1011\ 11_2 = 687$. With 16 frames, the PFN is 4 bits. Mapping VPN 687 to frame 9 gives $9 \times 1024 + 0x0DE = 9 \times 1024 + 222 = 9,438$, or `0x24DE`.
2. First fit uses 280 KiB and leaves (120, 90, 640, 200) KiB. Best fit uses 200 KiB and leaves (120, 280, 640, 10) KiB. Worst fit uses 640 KiB and leaves (120, 280, 450, 200) KiB. Best fit leaves the smallest selected-hole remainder: 10 KiB.

3. $P = 1101 \oplus 0011 \oplus 0101$. Since $1101 \oplus 0011 = 1110$ and $1110 \oplus 0101 = 1011$, parity is $P = 1011$. If D_2 fails, then $D_2 = D_0 \oplus D_1 \oplus P = 1101 \oplus 0011 \oplus 1011$. The first XOR is 1110, and $1110 \oplus 1011 = 0101$, recovering the original block.
4. The active translations can continually evict one another from a TLB whose capacity or associativity is too small, producing TLB misses even though every page-table entry is valid and resident. The hardware fact to investigate is TLB capacity/associativity; the access-pattern fact is whether the program cycles across more VPNs than the TLB can retain before reuse.

References

- [1] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.
- [2] William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 10th edition, 2015.